

KUBERNETES

HANDWRITTEN NOTES



*Your Success is
My Motivation* ❤️



KUBERNETES NOTES

♥ From Basics to Advanced Level ♥

≡ Introduction / Table of Contents ≡

S.No	Chapter Name	Pages
①	Cloud-Native and DevOps Fundamentals	12
②	Linux and Networking Prerequisites	12
③	Containers and Docker Basics	14
④	Introduction to Kubernetes	12
⑤	Kubernetes Architecture	14
⑥	Cluster Setup and Installation	14
⑦	kubectl and YAML Fundamentals	12
⑧	Kubernetes Objects, Labels, and Namespaces	14
⑨	Pods and Pod Lifecycle	16
⑩	ReplicaSets and Deployments	14
⑪	StatefulSets	12
⑫	DaemonSets, Jobs, and CronJobs	14
⑬	ConfigMaps and Secrets	12
⑭	Probes, Hooks, and Init Containers	14
⑮	Resource Management and QoS	14
⑯	Services and Service Discovery	14
⑰	Kubernetes Networking and DNS	16
⑱	Ingress and Gateway API	14
⑲	Network Policies	12
⑳	Storage: PV, PVC, and StorageClass	16
㉑	Scheduling, Affinity, and Taints	16
㉒	Autoscaling: HPA, VPA, and CA	14
㉓	Authentication, Authorization, and RBAC	16
㉔	Pod Security and Cluster Security	16
㉕	Cluster Administration and Upgrades	14
㉖	Monitoring, Logging, and Observability	16
㉗	Troubleshooting and Debugging	14
㉘	Helm and Kustomize	12
㉙	CI/CD and GitOps	14
㉚	CRDs, Controllers, and Operators	14
㉛	Production Kubernetes on EKS, AKS, and GKE	18
㉜	Projects and Interview Preparation	16

Learn
Kubernetes
Step by
Step

Practice
YAML
Makes
Perfect

Master
K8s from
Basics to
Advanced

Book Summary

- ♥ 32 Chapters
- ♥ Covers Beginner to Advanced Topics
- ♥ Includes Architecture, Workloads, Networking, Storage, and Security
- ♥ Includes kubectl Commands, YAML, Examples, Labs, and Diagrams
- ♥ Includes Helm, GitOps, and Cloud Kubernetes
- ♥ Includes Real-World Projects
- ♥ Includes Interview Questions and Troubleshooting
- ♥ Bonus: Mini Projects, Practice Questions, and Mock Interview
- ♥ Total ≈ 500 Pages



CHAPTER 4

Introduction to Kubernetes

1

1. What is Kubernetes?

- Kubernetes is an open-source platform for automating deployment, scaling, and management of containerized applications.
- It groups machines into a cluster and runs containers efficiently across them.
- It helps manage containers in production environments.
- It monitors application health and restarts failed containers.
- It can scale applications up or down based on need.
- It provides service discovery and load balancing.
- It supports declarative management using YAML manifests.
- It was originally designed by Google and is now maintained by the CNCF/community.

Automate,
scale,
manage!

Key idea:

Kubernetes = container orchestration platform

Chapter Summary

- ♥ **What it is:** Kubernetes is an open-source platform to run and manage containerized applications at scale.
- ♥ **Why it is required:** It solves the challenges of scaling, availability, resilience, and efficient management of containers.
- ♥ **Features:** Automation, self-healing, scaling, load balancing, service discovery, rolling updates, and more.
- ♥ **Orchestration:** It orchestrates containers across a cluster of nodes to keep applications running as desired.
- ♥ **Use cases:** Microservices, web apps, CI/CD pipelines, data processing, ML/AI workloads, and more.
- ♥ **Comparisons:** More powerful than Docker, Docker Compose, and Docker Swarm for production-scale environments.
- ♥ **Ecosystem & config:** Large ecosystem with tools and operators; uses declarative YAML manifests for desired state management.



kubernetes

2

2. Why Kubernetes is required

- Managing many containers manually becomes difficult.
- Applications need high availability and automatic recovery.
- Traffic changes require easy scaling.
- Deployments should be consistent across environments.
- Containers may run on many machines, so coordination is needed.
- Rolling updates and rollback should happen smoothly.
- Load balancing and service discovery are essential.
- Kubernetes reduces operational complexity in production.



Less
manual
work!

Problems without Kubernetes

- Manual scaling is slow and error-prone.
- Higher risk of downtime and failures.
- Networking between containers is hard.
- Monitoring and managing many containers is harder.

Need:

Kubernetes is needed to automate container management at scale.

3

3. Kubernetes features

- Automated deployment and scaling
- Self-healing of failed containers
- Service discovery and load balancing
- Rolling updates and rollbacks
- Storage orchestration
- Secret and configuration management
- Efficient resource usage
- Horizontal scaling and high availability
- Batch execution support

Smart
automation!

Popular features

- **Declarative system:** Define the desired state, Kubernetes makes it happen.
- **Self-healing:** Automatically restarts or replaces failed containers.
- **Service abstraction:** Exposes apps behind stable services.
- **Rolling updates:** Update apps with zero downtime.
- **Config & secrets:** Manage settings and sensitive data securely.
- **Storage orchestration:** Attach and manage persistent storage.
- **Resource management:** Optimal use of CPU, memory, and storage.
- **High availability:** Apps stay available even if nodes fail.

Core strengths

- ★ **Automation:** Reduces manual work and errors.
- ★ **Resilience:** Keeps apps running even during failures.
- ★ **Scalability:** Handles growth with ease.
- ★ **Portability:** Runs everywhere—cloud, on-prem, or hybrid.

4

4. Container orchestration

- Container orchestration means managing many containers automatically.
- It handles scheduling, scaling, networking, health checks, and recovery.
- It decides where containers should run in a cluster.
- It ensures the desired number of instances stay running.
- It restarts failed containers and replaces unhealthy ones.
- It connects containers with networking and services.

Manage
many
containers
easily!



Tasks of orchestration

- **Deploy:** Create and deploy containers.
- **Schedule:** Decide where containers will run.
- **Monitor:** Check the health and status.
- **Scale:** Increase or decrease the number of containers.
- **Heal:** Restart, replace, and recover containers.
- **Network:** Connect containers and expose services.
- **Update:** Roll out updates and roll back if needed.

Orchestration flow



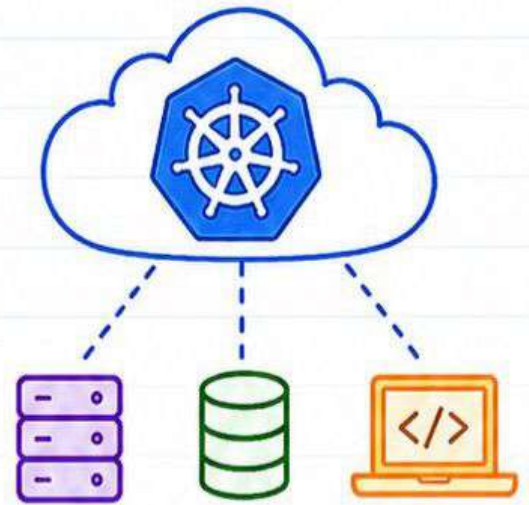
In short

Orchestration = automated coordination of containers.

5

5. Kubernetes use cases

- Microservices applications
- Web applications and APIs
- CI/CD deployment environments
- Big data and batch processing
- Machine learning / AI workloads
- Hybrid and multi-cloud applications
- Dev, test, and staging environments
- High-availability production systems



Used
everywhere!

Real-world examples

- ♥ **E-commerce app:** Handles traffic spikes, updates, and scaling with ease.
- ♥ **Streaming app:** Delivers content reliably across many users and regions.
- ♥ **Banking app:** Ensures high availability, security, and compliance.
- ♥ **Analytics platform:** Processes large datasets and scales compute jobs.

Best suited for

- Scalable applications
- Distributed systems and services
- Modern cloud-native workloads



6. Kubernetes versus Docker

Kubernetes and Docker are not direct replacements; they solve different problems.

- Docker is a container platform/tool used to build and run containers.
- Kubernetes is an orchestration platform used to manage containers at scale.
- Docker focuses on single-container or small local workflows.
- Kubernetes focuses on clusters, scaling, networking, and automation.
- Docker is simpler to start with.
- Kubernetes is more powerful for production environments.

Build
vs
manage!

Aspect	Docker	Kubernetes
Purpose	Build, package, and run containers.	Orchestrate and manage containers at scale.
Scope	Works on a single machine (local or cloud).	Works across a cluster of machines (nodes).
Scaling	Manual scaling, limited.	Automatic scaling and self-healing.
Networking	Basic networking for containers.	Advanced networking across pods and services.
Learning curve	Easy to learn and start.	Steeper learning curve, more concepts.
Best use	Developing, testing, and running individual containers.	Production environments that need scale, reliability, and automation.

Remember

Docker creates containers; Kubernetes manages many containers.



7

7. Kubernetes versus Docker Compose

- Docker Compose defines and runs multi-container apps on one machine.
- Kubernetes manages containerized apps across clusters of machines.
- Compose is lightweight and easy for local development.
- Kubernetes is more advanced and suitable for production.
- Compose uses a compose YAML file; Kubernetes uses multiple resource manifests.
- Compose offers simpler setup; Kubernetes offers more features and resilience.

Dev vs
production!



	Docker Compose	Kubernetes
Environment	Runs on a single machine	Runs across a cluster of machines
Scale	Limited to one host	Built to scale across many nodes
Setup	Very easy to install and use	More complex to set up and manage
High availability	Not built-in	Built-in with self-healing, replication, failover
Networking	Simple, local networking between containers	Advanced networking across nodes and services
Best for	Local development, testing, small projects	Production, large-scale, highly available apps

Best use:

Compose for local/dev;

Kubernetes for large-scale production.



8. Kubernetes versus Docker Swarm

- Docker Swarm is Docker's native orchestration tool.
- Kubernetes is a more feature-rich orchestration platform.
- Swarm is easier to set up and simpler to learn.
- Kubernetes has a larger ecosystem and broader adoption.
- Kubernetes supports more advanced scaling, scheduling, and self-healing features.
- Swarm can suit smaller setups; Kubernetes suits complex production workloads.

Simple
vs
powerful!

Aspect	Docker Swarm	Kubernetes
Setup	Easy to set up; comes built into Docker.	More components; setup is more involved.
Features	Basic orchestration features.	Rich feature set: auto-scaling, self-healing, rollouts, more.
Ecosystem	Smaller ecosystem.	Large ecosystem with many tools and integrations.
Scalability	Good for small to medium scale.	Excellent for large-scale and complex environments.
Complexity	Simple to learn and manage.	More complex; steeper learning curve.
Best for	Small teams, simple apps, and lightweight needs.	Enterprise apps, complex workloads, and large teams.

Quick idea:

Swarm is simpler; Kubernetes is more powerful.

9. Kubernetes ecosystem

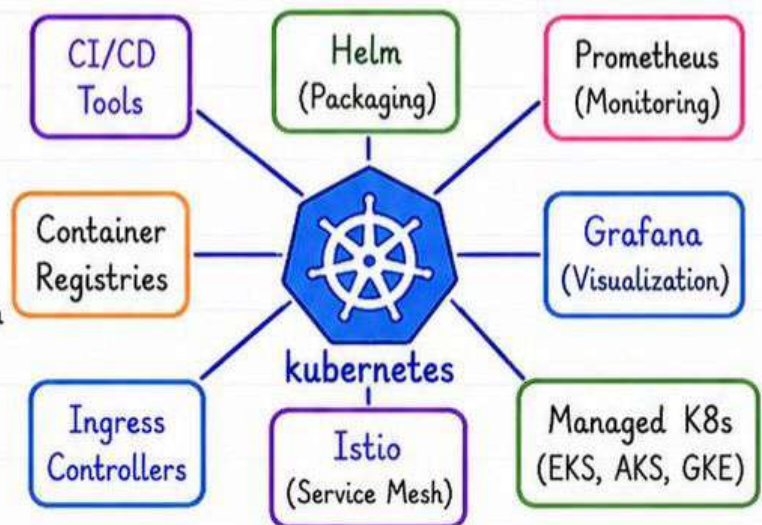
Kubernetes does not work alone. It is part of a rich ecosystem of tools, platforms, and surrounding components that extend its capabilities and make it production-ready.

- CNCF and open-source community support drive innovation, standards, and collaboration.
- kubectl is the official command-line tool for managing clusters.
- Helm helps in package management and application templating.
- Prometheus and Grafana are widely used for monitoring and visualization.
- Istio provides service mesh capabilities like traffic management, security, and observability.
- Ingress controllers manage external access to services.
- Container registries store and distribute container images.
- CI/CD tools integrate with Kubernetes for automated delivery.
- Managed Kubernetes services like EKS, AKS, and GKE simplify cluster operations.

Many tools,
one platform!

Common ecosystem tools

- ♥ kubectl - command-line interface
- ♥ Helm - package manager
- ♥ Prometheus - monitoring
- ♥ Grafana - dashboards & visualization
- ♥ Istio - service mesh
- ♥ NGINX Ingress - ingress controller
- ♥ Docker Hub - container registry
- ♥ Jenkins - CI/CD integration



Why ecosystem matters

It extends Kubernetes for deployment, monitoring, security, and operations.

- ♥ CI/CD Tools: automate build, test, and deploy.
- ♥ Helm: install, upgrade, and manage applications.
- ♥ Prometheus: collect metrics and alert.
- ♥ Grafana: visualize metrics and create dashboards.
- ♥ Istio: manage traffic, security, and policies.
- ♥ Ingress Controllers: route external traffic.
- ♥ Registries: store and pull container images.
- ♥ Managed K8s: cloud providers handle the infrastructure.

10

10. Declarative configuration

- Declarative configuration means describing the desired state of the system.
- In Kubernetes, YAML manifests are used to declare resources.
- You describe what should run, not every step to run it.
- Kubernetes continuously works to match actual state with desired state.
- This makes management repeatable, version-controlled, and consistent.
- Declarative style supports automation and GitOps practices.

Describe
the goal!

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
```

Key point

Desired state is central to Kubernetes.

Imperative vs declarative

<u>Imperative</u>	<u>Declarative</u>
• You tell Kubernetes exactly what steps to take. • Example: <code>kubectl run</code>, <code>kubectl create</code>, <code>kubectl scale</code> ... • Focus is on commands and actions.	• You describe the final desired state. • Example: YAML manifest files. • Focus is on what should exist, not how to do it.